



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных Технологий
Кафедра Прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине
«Технологии и инструментарий анализа больших данных»

Студент группы: ИКБО-04-22

Кликушин В.И.
(Ф. И.О. студента)

Преподаватель

Царев Р.Ю.
(Ф.И.О. преподавателя)

Москва 2025

СОДЕРЖАНИЕ

1 Практическая часть	3
1.1 Задание №1	3
1.2 Задание №2	3
1.3 Задание №3	4
1.4 Задание №4	5
1.5 Задание №5	6
1.6 Задание №6	7
1.7 Задание №7	9
1.8 Задание №8	10
1.9 Задание №9	10
1.10 Задание №10	11
1.11 Задание №11	12
1.12 Задание №12	13

1 ПРАКТИЧЕСКАЯ ЧАСТЬ

1.1 Задание №1

Определить два вектора, представляющие собой число автомобилей, припаркованных в течение 5 рабочих дней у бизнес-центра на уличной стоянке и в подземном гараже.

Код инициализации двух векторов представлен в Листинге А.

Листинг А – Реализация задания №1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

sns.set_theme(style="whitegrid")

data = {
    "Day": ["Mon", "Tue", "Wed", "Thu", "Fri"],
    "Street": [80, 98, 75, 91, 78],
    "Garage": [100, 82, 105, 89, 102]
}
df_parking = pd.DataFrame(data)
df_parking
```

Заданный датафрейм представлен на Рисунке 1.1.1.

	Day	Street	Garage
0	Mon	80	100
1	Tue	98	82
2	Wed	75	105
3	Thu	91	89
4	Fri	78	102

Рисунок 1.1.1 – Определение датафрейма

1.2 Задание №2

Найти и интерпретировать корреляцию между переменными «Улица» и «Гараж» (подсчитать корреляцию по Пирсону).

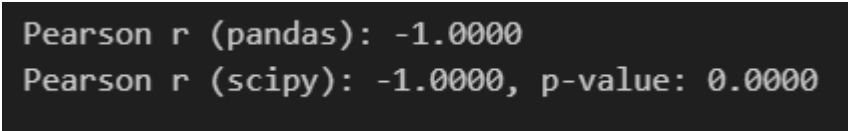
Определение коэффициента корреляции представлено в Листинге Б.

Листинг Б – Реализация задания №2

```
pearson_r = df_parking["Street"].corr(df_parking["Garage"])
r, pvalue = stats.pearsonr(df_parking["Street"], df_parking["Garage"])

print(f"Pearson r (pandas): {pearson_r:.4f}")
print(f"Pearson r (scipy): {r:.4f}, p-value: {pvalue:.4f}")
```

Посчитанный коэффициент отображен на Рисунке 1.2.1.



```
Pearson r (pandas): -1.0000
Pearson r (scipy): -1.0000, p-value: 0.0000
```

Рисунок 1.2.1 – Определение коэффициента корреляции

1.3 Задание №3

Построить диаграмму рассеяния для вышеупомянутых переменных.

Код для построения диаграммы рассеяния представлен в Листинге В.

Листинг В – Код для построения диаграммы рассеяния

```
plt.figure(figsize=(6,5))
sns.scatterplot(x="Street", y="Garage", data=df_parking, color="royalblue")

for i, row in df_parking.iterrows():
    plt.text(row["Street"] + 0.5, row["Garage"] + 0.5, row["Day"],
             fontsize=10, color="darkred", weight="bold")

plt.title("Диаграмма рассеяния: число автомобилей на улице и в гараже", fontsize
= 12)
plt.xlabel("Число автомобилей на улице", fontsize = 10)
plt.ylabel("Число автомобилей в гараже", fontsize = 10)
plt.grid(True)
plt.tight_layout()
plt.show()
```

Построенная диаграмма рассеяния представлена на Рисунке 1.3.1.

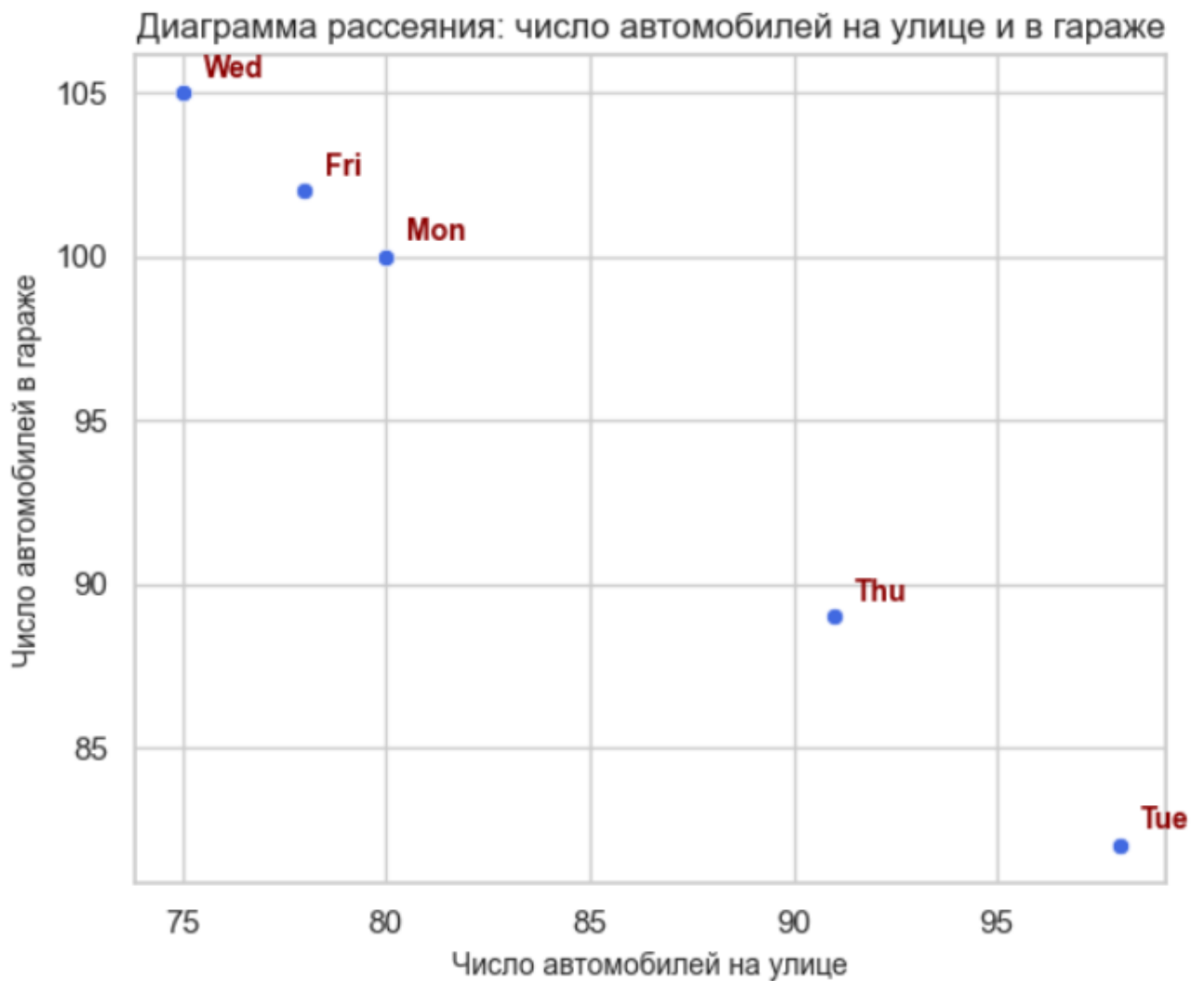


Рисунок 1.3.1 – Построенная диаграмма рассеяния

1.4 Задание №4

Найти и выгрузить данные. Вывести, провести предобработку и описать признаки.

Код для загрузки и предобработки датасета представлен в Листинге Г.

Листинг Г – Код для загрузки и предобработки датасета

```
import kagglehub
import pandas as pd
import os

path = kagglehub.dataset_download("ekibee/car-sales-information")

file_path = os.path.join(path, "all_regions.csv")
df = pd.read_csv(file_path)

n = 5
df.head(n)
df.info()
```

Продолжение Листинга Г

```
df.duplicated().sum()
df.isna().sum()
df.dropna(how="any", inplace=True, ignore_index=True)
df.isna().sum()
df.shape
df.drop(columns=["link", "parse_date"], inplace=True)
df.head()
df["year"] = df["year"].astype("int64")
df["mileage"] = df["mileage"].astype("int64")
df["power"] = df["power"].astype("int64")
df["engineDisplacement"] =
df["engineDisplacement"].str.rstrip("LTR").astype("float64")
df["date"] = df["date"].astype("datetime64[ns]")
df.head()
df.info()
```

Предобработанный датасет отображен на Рисунке 1.4.1.

	brand	name	bodyType	color	fuelType	year	mileage	transmission	power	price	vehicleConfigurati
0	Лада	1111 Ока	Хэтчбек 3 дв.	Фиолетовый	Бензин	1996	37000	Механика	33	95000	0.6 MT 111
1	Toyota	Funcargo	Хэтчбек 5 дв.	Серебристый	Бензин	2002	295000	АКПП	87	380000	1.3
2	Лада	Гранта	Лифтбек	Серый	Бензин	2018	16000	Механика	106	795000	1.6 MT Comfc
3	Chevrolet	Niva	Джип 5 дв.	Зеленый	Бензин	2002	220000	Механика	80	450000	1.7 MT
4	Лада	2114 Самара	Хэтчбек 5 дв.	Черный	Бензин	2007	185000	Механика	81	120000	1.6 MT 21144-22-0

Рисунок 1.4.1 – Предобработанный датасет

1.5 Задание №5

Построить корреляционную матрицу по одной целевой переменной. Определить наиболее коррелирующую переменную, продолжить с ней работу в следующем пункте.

Код для построения корреляционной матрицы представлен в Листинге Д.

Листинг Д – Код для построения корреляционной матрицы

```
target_col = 'price'
if target_col not in df.columns:
    raise ValueError(f"Целевая переменная '{target_col}' не найдена. Проверь название столбца в DataFrame.")
```

Продолжение Листинга Д

```
corr_matrix = df.corr(numeric_only=True)
corr_matrix.style.background_gradient(cmap="coolwarm")

corr_price = df.corr(numeric_only=True).price.to_frame()
corr_price.style.background_gradient(cmap="coolwarm")
```

Построенная матрица представлена на Рисунке 1.5.1.

	year	mileage	power	price	engineDisplacement
year	1.000000	-0.514805	0.171553	0.489916	-0.010728
mileage	-0.514805	1.000000	0.012477	-0.362176	0.119452
power	0.171553	0.012477	1.000000	0.569977	0.849239
price	0.489916	-0.362176	0.569977	1.000000	0.426155
engineDisplacement	-0.010728	0.119452	0.849239	0.426155	1.000000

Рисунок 1.5.1 – Корреляционная матрица

1.6 Задание №6

Реализовать регрессию вручную, отобразить наклон, сдвиг и MSE.

Код реализации линейной регрессии представлен в Листинге Е.

Листинг Е – Код реализации линейной регрессии

```
def gradient_descent(X, y, lr=0.001, eps=1e-10, max_iter=10000):
    """
    Реализация градиентного спуска для линейной регрессии
    """
    X_mean, X_std = np.mean(X), np.std(X)
    y_mean, y_std = np.mean(y), np.std(y)

    X_normalized = (X - X_mean) / X_std
    y_normalized = (y - y_mean) / y_std

    k = 0
    b = 0
    n = len(y_normalized)

    prev_loss = None

    for i in range(max_iter):
        y_pred = k * X_normalized + b

        loss = np.mean((y_normalized - y_pred) ** 2)

        grad_k = (-2/n) * np.sum((y_normalized - y_pred) * X_normalized)
        grad_b = (-2/n) * np.sum(y_normalized - y_pred)

        k -= lr * grad_k
        b -= lr * grad_b
```

Продолжение Листинга E

```
        if prev_loss is not None and abs(prev_loss - loss) < eps:
            print(f"Сходимость достигнута на итерации {i}")
            break

        prev_loss = loss

    if i % 1000 == 0:
        print(f"Iteration {i}, Loss: {loss:.6f}")

    k_original = k * (y_std / X_std)
    b_original = b * y_std + y_mean - k * (y_std / X_std) * X_mean

    return k_original, b_original

k_original, b_original = gradient_descent(X, y)
gradient_descent_model = k_original * X + b_original

print(f"k = {k_original}")
print(f"b = {b_original}")

plt.figure(figsize=(10, 6))

plt.scatter(X, y, alpha=0.5)

plt.plot(X, model_skllearn, color='red', linewidth=2,
         label=f'Skllearn (y={model.coef_[0][0]:.2f}x + {model.intercept_[0]:.2f})')

plt.plot(X, gradient_descent_model, color='green', linewidth=2,
         label=f'Градиентный спуск (y={k_original:.2f}x + {b_original:.2f})')

plt.xlabel('Мощность (л.с.)', fontsize=12)
plt.ylabel('Цена (руб.)', fontsize=12)
plt.title('Сравнение моделей линейной регрессии: Sklearn LinearRegression и\nГрадиентный спуск', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Построенная модель регрессии представлена на Рисунке 1.6.1.

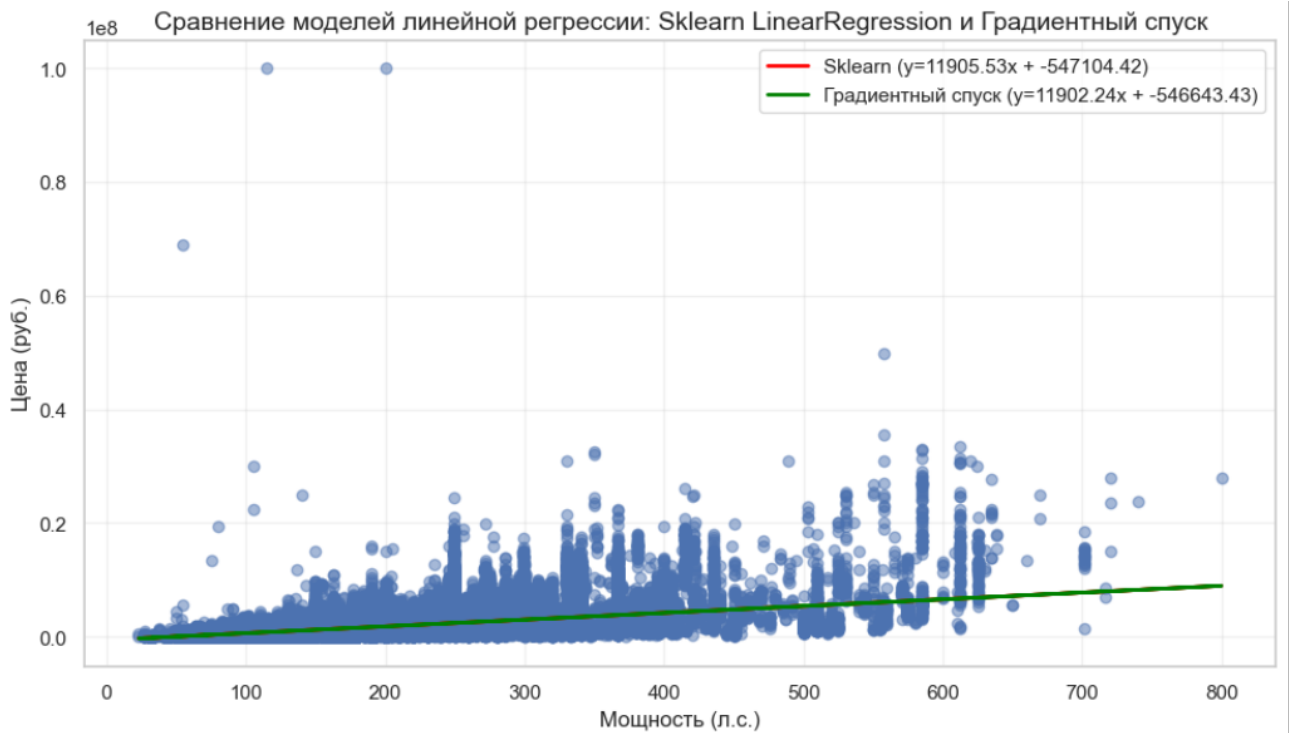


Рисунок 1.6.1 – Построенная модель регрессии

1.7 Задание №7

Загрузить данные: «insurance.csv». Вывести и провести предобработку. Вывести список уникальных регионов.

Код загрузки и предобработки данных представлен в Листинге Ж.

Листинг Ж – Код загрузки и предобработки данных

```
import os
insurance_path = "insurance.csv"

if not os.path.exists(insurance_path):
    raise FileNotFoundError(
        "Файл 'insurance.csv' не найден в текущей директории. "
        "Положите 'insurance.csv' в рабочую папку ноутбука и перезапустите "
        "ячейку."
    )

df_ins = pd.read_csv(insurance_path)
df_ins.head()

print("Shape:", df_ins.shape)
print("Нулевые значения в каждом столбце:\n", df_ins.isnull().sum())
print("Уникальные регионы:", df_ins['region'].unique())
```

Информация о датасете отображена на Рисунке 1.7.1.

```

Shape: (1338, 7)
Нулевые значения в каждом столбце:
  age      0
  sex      0
  bmi      0
  children 0
  smoker   0
  region   0
  charges  0
dtype: int64
Уникальные регионы: ['southwest' 'southeast' 'northwest' 'northeast']

```

Рисунок 1.7.1 – Информация о датасете

1.8 Задание №8

Выполнить однофакторный ANOVA тест, чтобы проверить влияние региона на индекс массы тела (BMI), используя первый способ, через библиотеку Scipy.

Код для выполнения однофакторного ANOVA теста представлен в Листинге 3.1.

Листинг 3.1 – Код для выполнения однофакторного ANOVA теста

```

groups = [group["bmi"].values for _, group in df_ins.groupby("region")]
f_stat, pval = stats.f_oneway(*groups)
print(f"ANOVA (scipy) region -> BMI: F = {f_stat:.4f}, p-value = {pval:.6f}")

```

Результат выполнения теста представлен на Рисунке 1.8.1.

```

.. ANOVA (scipy) region -> BMI: F = 39.4951, p-value = 0.000000

```

Так как p-value < 0.05, нулевая гипотеза отвергается, значит средние значения BMI значимо различаются между регионами. Регион влияет на индекс массы тела.

Рисунок 1.8.1 - Результат выполнения теста

1.9 Задание №9

Выполнить однофакторный ANOVA тест, чтобы проверить влияние региона на индекс массы тела (BMI), используя второй способ, с помощью функции `anova_lm()` из библиотеки `statsmodels`.

Код для выполнения однофакторного ANOVA теста с помощью функции `anova_lm()` из библиотеки `statsmodels` представлен в Листинге И.

Листинг И – Код для выполнения однофакторного ANOVA теста с помощью функции `anova_lm()` из библиотеки `statsmodels`

```
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.anova import anova_lm

model = ols("bmi ~ region", data=df_ins).fit()
anova_table = anova_lm(model, typ=2)
anova_table
```

Результат выполнения однофакторного ANOVA теста с помощью функции `anova_lm()` из библиотеки `statsmodels` представлен на Рисунке 1.9.1.

	sum_sq	df	F	PR(>F)
region	4055.880631	3.0	39.495057	1.881839e-24
Residual	45664.319755	1334.0	NaN	NaN

Sum of Squares — сумма квадратов отклонений

Degrees of Freedom — степени свободы (сколько независимых значений участвует в оценке)

F-statistic — отношение межгрупповой дисперсии к внутригрупповой

p-value — вероятность получить такое (или большее) значение F при H0 (если < 0.05 — различия значимы)

Рисунок 1.9.1 – Результат выполнения однофакторного ANOVA теста с помощью функции `anova_lm()` из библиотеки `statsmodels`

1.10 Задание №10

С помощью t критерия Стьюдента перебрать все пары. Определить поправку Бонферрони. Сделать выводы.

Реализация задания представлена в Листинге К.

Листинг К – Реализация задания №10

```
from itertools import combinations

regions = df_ins['region'].unique()
pairs = list(combinations(regions, 2))
alpha = 0.05
m = len(pairs)
alpha_bonf = alpha / m

results = []
for a, b in pairs:
    a_vals = df_ins[df_ins['region'] == a]['bmi']
    b_vals = df_ins[df_ins['region'] == b]['bmi']
```

Продолжение Листинга К

```
t, p = stats.ttest_ind(a_vals, b_vals, equal_var=False)
results.append((a, b, t, p, p < alpha_bonf))

res_df = pd.DataFrame(results,
columns=["region_a", "region_b", "t", "p", "reject_H0_after_Bonf"])
res_df_sorted = res_df.sort_values("p")
res_df_sorted
```

Результат статического теста показан на Рисунке 1.10.1.

	region_a	region_b	t	p	reject_H0_after_Bonf
3	southeast	northwest	9.376833	9.989831e-20	True
4	southeast	northeast	8.835471	8.311733e-18	True
0	southwest	southeast	-5.951605	4.233221e-09	True
1	southwest	northwest	3.284417	1.077543e-03	True
2	southwest	northeast	3.116697	1.910080e-03	True
5	northwest	northeast	0.060294	9.519402e-01	False

Рисунок 1.10.1 - Результат статического теста

1.11 Задание №11

Выполнить пост-хок тесты Тьюки и построить график.

Код выполнения пост-хок тестов Тьюки представлен в Листинге Л.

Листинг Л – Реализация задания №11

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

tukey = pairwise_tukeyhsd(endog=df_ins['bmi'], groups=df_ins['region'],
alpha=0.05)
tukey.summary()
fig = tukey.plot_simultaneous(figsize=(8,6))
plt.title("Tukey HSD: BMI by Region")
plt.show()
```

Результат выполнения пост-хок тестов Тьюки представлен на Рисунке 1.11.1.

Multiple Comparison of Means - Tukey HSD, FWER=0.05						
group1	group2	meandiff	p-adj	lower	upper	reject
northeast	northwest	0.0263	0.9999	-1.1552	1.2078	False
northeast	southeast	4.1825	0.0	3.033	5.332	True
northeast	southwest	1.4231	0.0107	0.2416	2.6046	True
northwest	southeast	4.1562	0.0	3.0077	5.3047	True
northwest	southwest	1.3968	0.0127	0.2162	2.5774	True
southeast	southwest	-2.7594	0.0	-3.9079	-1.6108	True

Рисунок 1.11.1 - Результат выполнения пост-хок тестов Тьюки

1.12 Задание №12

Выполнить двухфакторный ANOVA тест, чтобы проверить влияние региона и пола на индекс массы тела (BMI), используя функцию `anova_lm()` из библиотеки `statsmodels`.

Код выполнения двухфакторного ANOVA теста представлен в Листинге М.

Листинг М – Код выполнения двухфакторного ANOVA теста

```
model2 = ols("bmi ~ region * sex", data=df_ins).fit()
anova_table2 = anova_lm(model2, typ=2)
anova_table2
```

Результат выполнения двухфакторного ANOVA теста представлен на Рисунке 1.12.1.

	sum_sq	df	F	PR(>F)
region	4034.975135	3.0	39.398134	2.163195e-24
sex	86.007035	1.0	2.519359	1.126940e-01
region:sex	174.157808	3.0	1.700504	1.650655e-01
Residual	45404.154911	1330.0	NaN	NaN

Рисунок 1.12.1 – Результат выполнения двухфакторного ANOVA теста